



FUNDAMENTOS E TÉCNICAS EM CIÊNCIAS DE DADOS

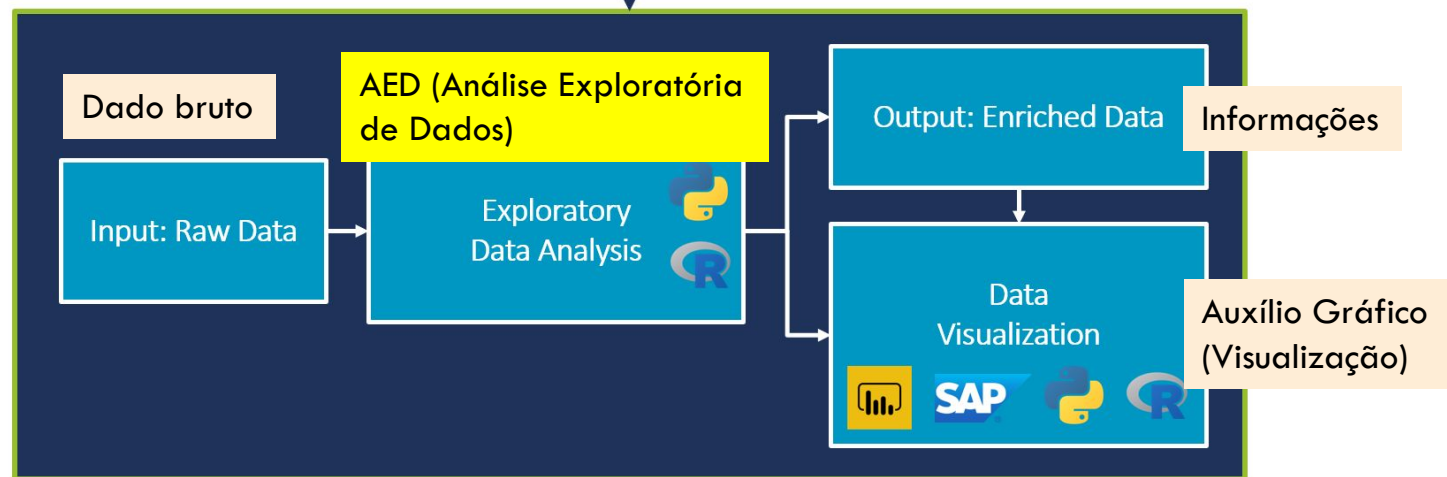
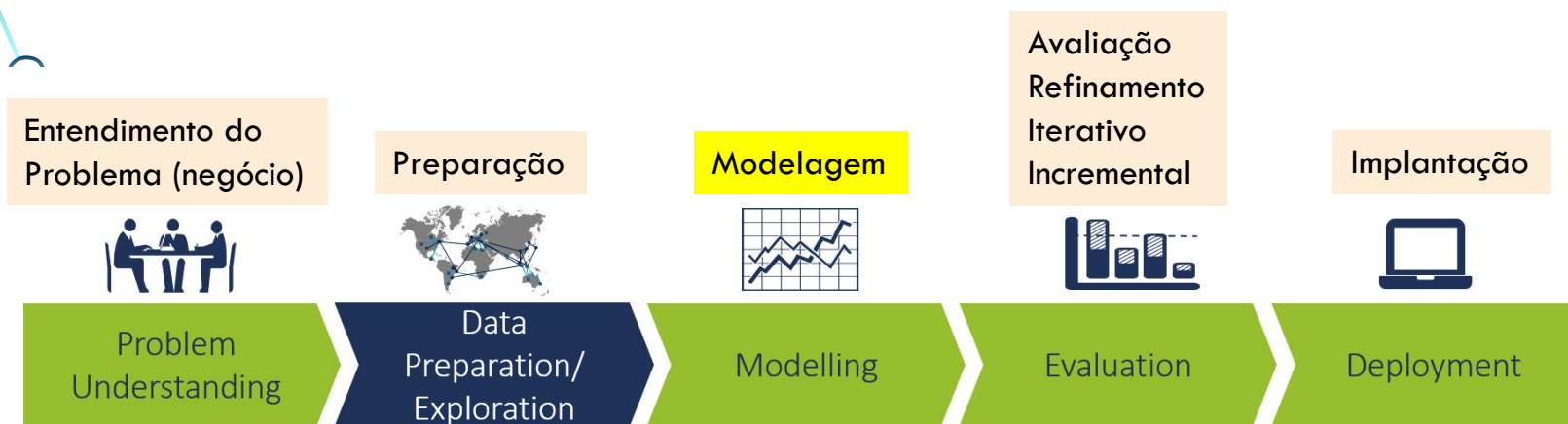
PROF. JOSENALDE OLIVEIRA

josenalde.oliveira@ufrn.br

<https://github.com/josenalde/datascience>

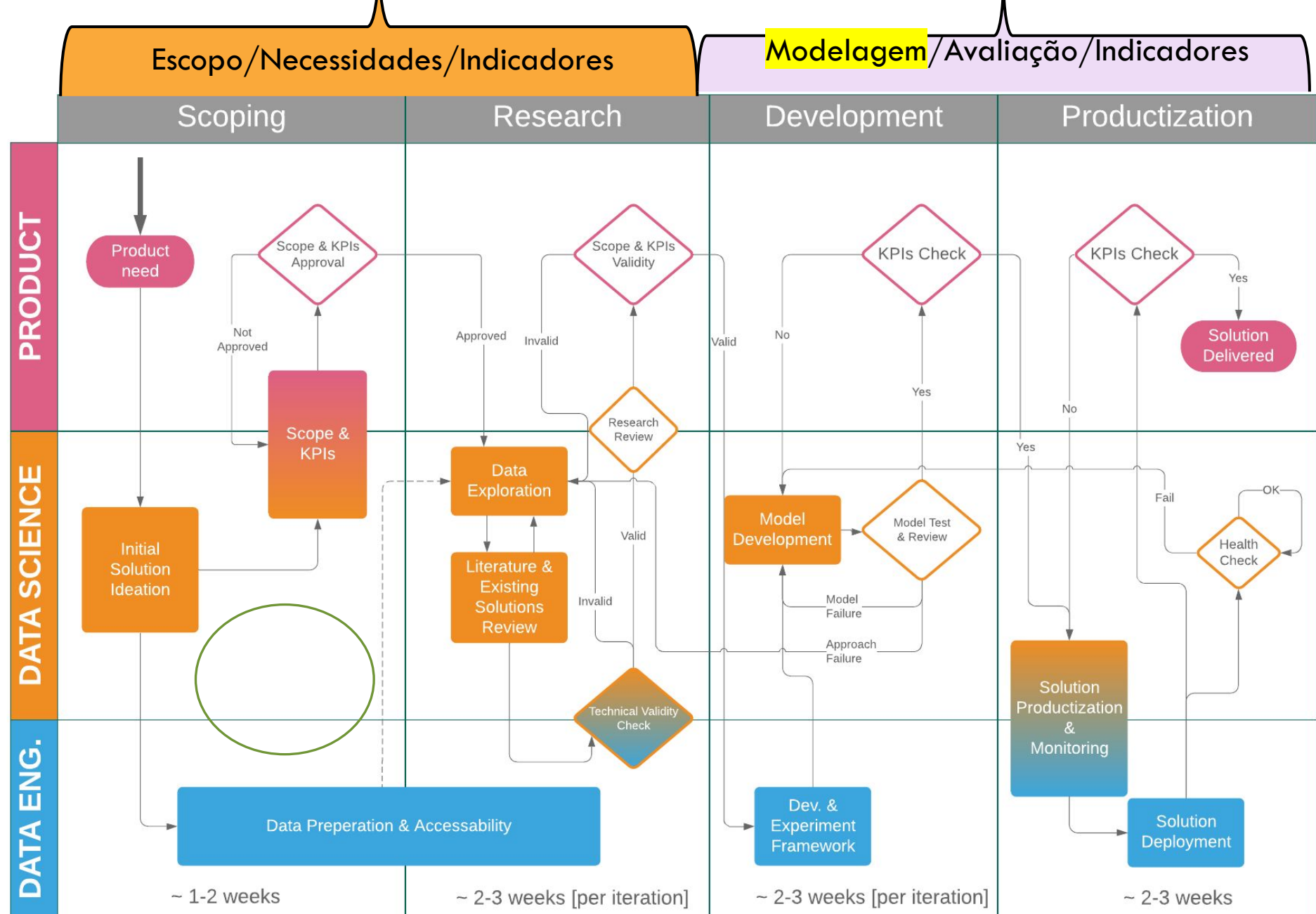
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - UFRN

NO QUE CONSISTE UM PROJETO DE CIÊNCIA DE DADOS?



Analisar dados – aplicar algum tipo de transformação nos dados em busca de conhecimento. Dados podem ser produzidos para posterior análise. A ideia é conhecer antes de analisar, para melhor escolha das técnicas.

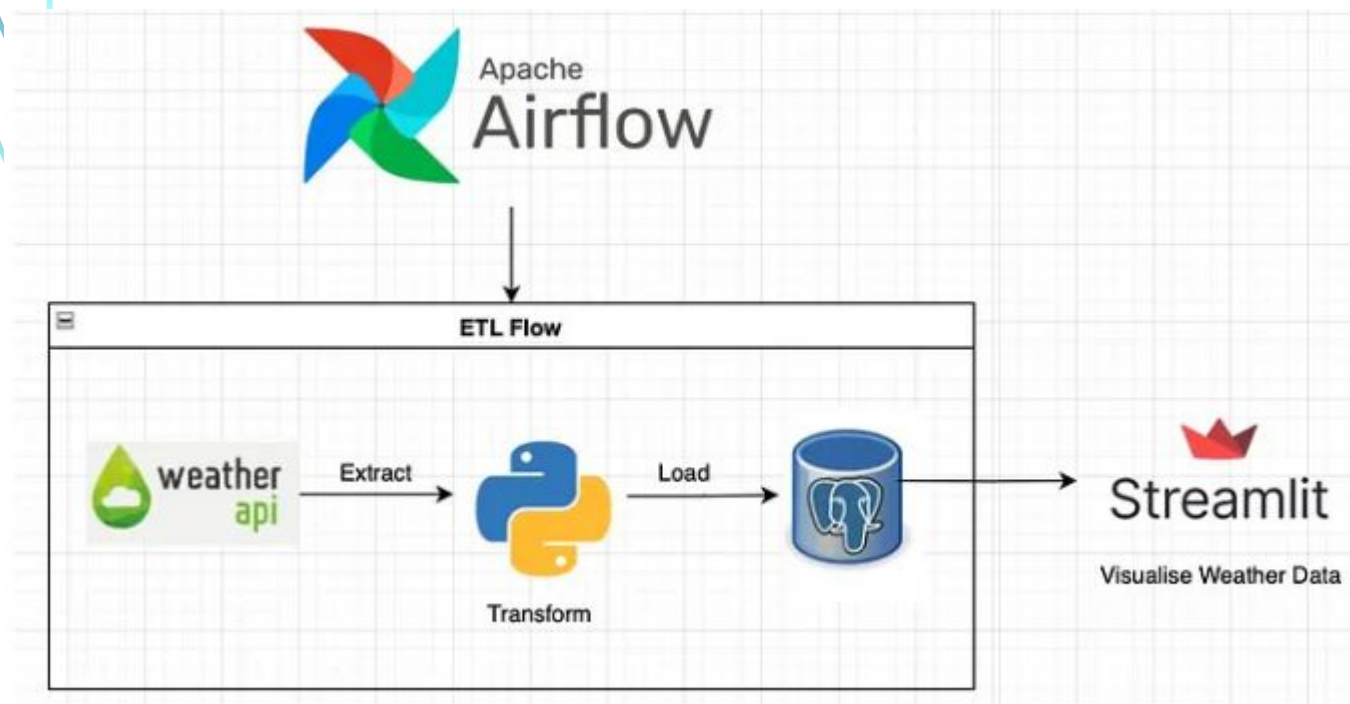
Exploratória: conhece os dados que irá analisar? Tem noção de como estão distribuídos? Quais suas médias? Desvios padrões? Como estão relacionados? Existem valores anormais?



A forma mais simples de entender a diferença é pela analogia da encanação e da água: o **Engenheiro de Dados constrói a tubulação** (infraestrutura, coleta e organização), enquanto o **Cientista de Dados usa a água que sai da torneira** (análise, estatística e modelos de Inteligência Artificial).

Aspecto	Engenharia de Dados	Ciência de Dados
Foco Principal	Infraestrutura e Disponibilidade. Garantir que os dados estejam limpos, acessíveis e em tempo real.	Análise e Predição. Extrair <i>insights</i> de negócios e prever comportamentos futuros.
Pergunta Central	<i>"Como coletar, armazenar e processar esses dados de forma escalável e segura?"</i>	<i>"O que esses dados nos dizem sobre o negócio e como prever a variável X?"</i>
Entregáveis	Pipelines de dados (ETL/ELT), Data Warehouses, Data Lakes, APIs de dados.	Relatórios analíticos, modelos de Machine Learning/IA, Dashboards estatísticos.
Público Atendido	Outros desenvolvedores, Cientistas de Dados e Analistas.	Tomadores de decisão, gerentes de produto e diretores de negócio.

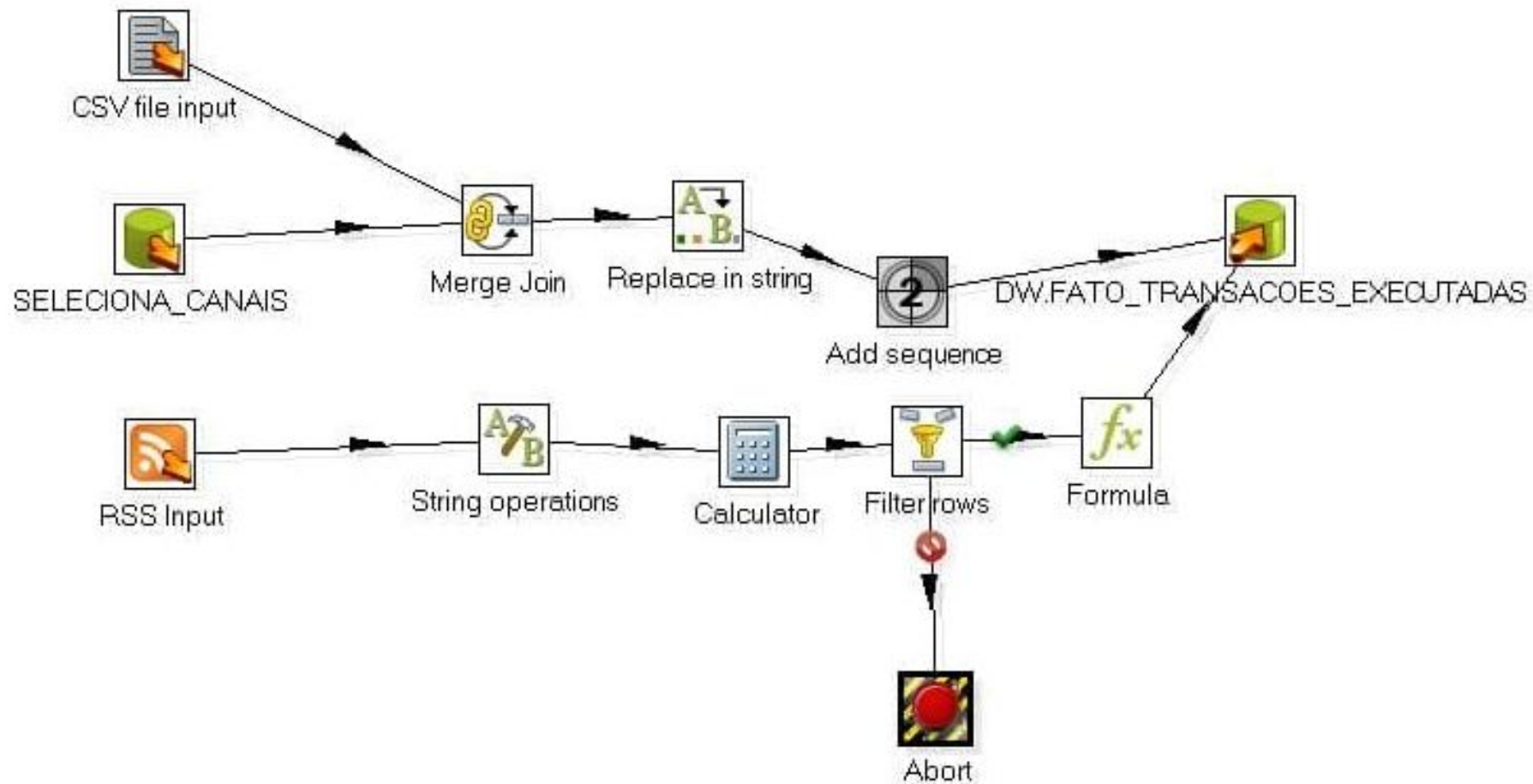
Exemplo de fluxo ETL (Extração, Transformação, Carga (Load)) orquestrada



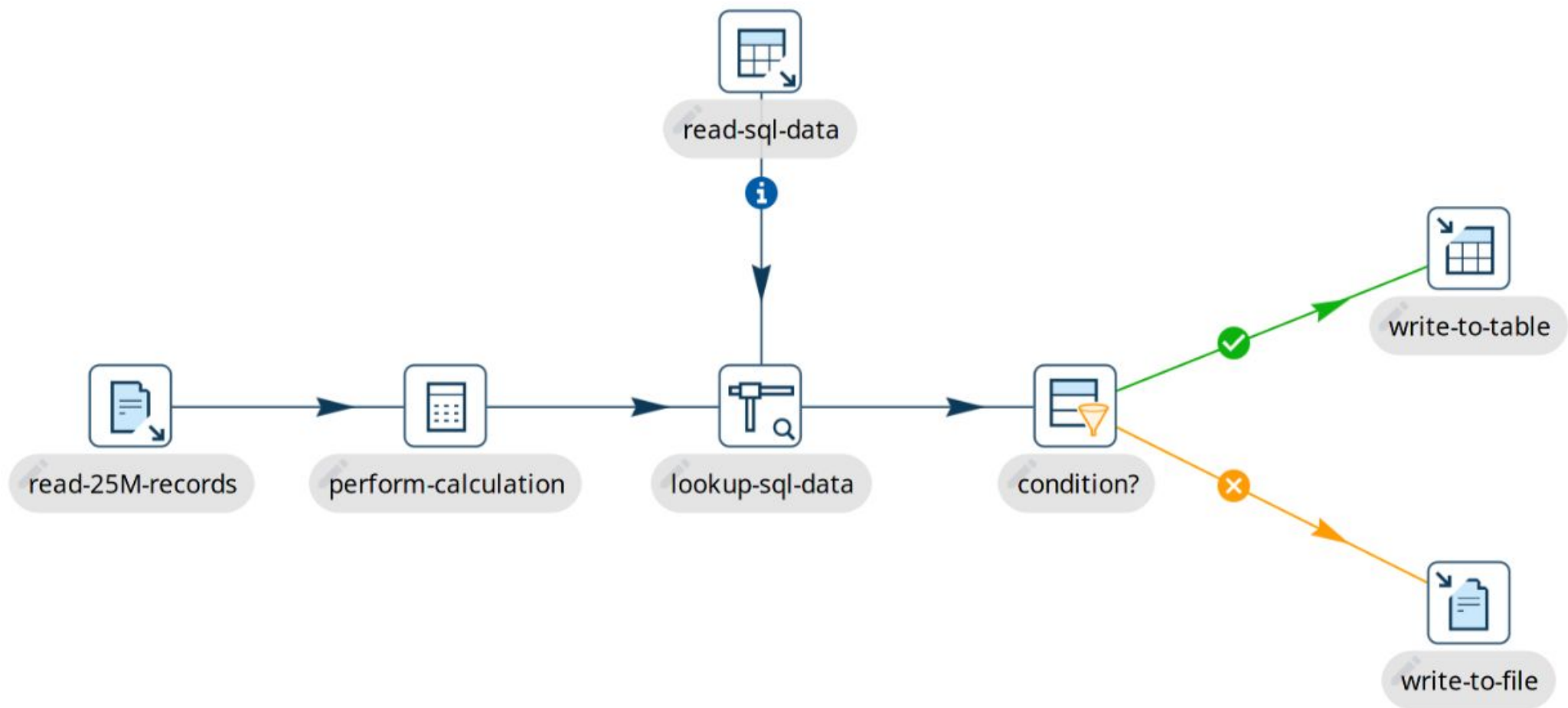
[Docs Airflow](#)

Fonte: [\[URL\]](#)

Ferramentas Pentaho Data Integration (Kettle)



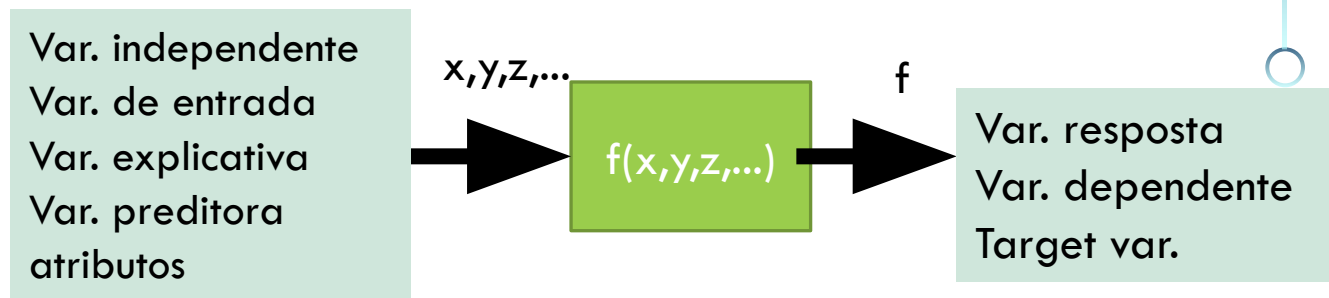
Ferramentas Apache Hop - ETL



O CONSUMO DESTES PIPELINES PODE SER POR MODELOS DE PREDIÇÃO

Mas o que é um modelo?

Simplesmente uma especificação de relação matemática (ou probabilística) existente entre variáveis diferentes



- Lucro nos próximos anos
- Probabilidade de ganhar
- Mensagem é spam ou não
- Transação no cartão é fraudulenta?
- Probabilidade de um banner ser clicado
- Que time ganhará?
- Impacto de ação A ou B em processo X

AS VÁRIAS POSSIBILIDADES DE MODELOS

Var Resposta	Var Indep.	Categóricas	Intervalares (numéricas)	Intervalares e categóricas
Intervalar (numérica)		ANOVA Árvore Decisão Random Forest Gradient boosting Rede neural	Regressão linear* Árvore decisão Random Forest Gradient boosting Rede neural	ANCOVA Árvore decisão Random Forest Gradient boosting Rede neural
Categórica		Regressão logística Rule induction Árvore Decisão Random Forest Gradient boosting Rede neural	Regressão logística Rule induction Árvore Decisão Random Forest Gradient boosting Rede neural	Regressão logística Rule induction Árvore Decisão Random Forest Gradient boosting Rede neural

Variáveis:

Qualitativas

Nominal (categoria, descrição, classe), exemplo:

cor dos olhos, tipo de material, gênero, modo de pagamento...

Ordinal (pequeno, médio, grande etc.)

Quantitativas

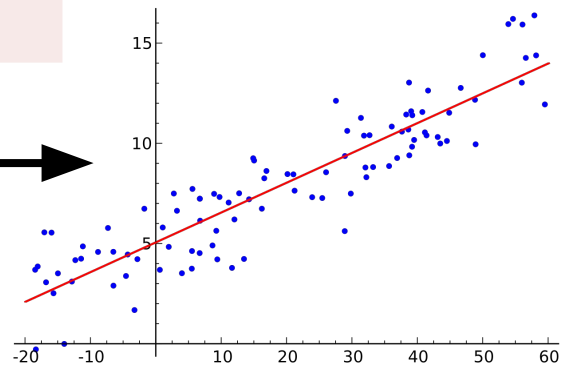
Discreta: têm um número contável de valores entre quaisquer dois valores. Uma variável discreta é sempre numérica. Por exemplo, o número de reclamações de clientes ou o número de falhas ou defeitos.

Contínua: medições (-inf, +inf)

*RL – preço de produtos variando em função da quantidade vendida

**ANOVA – gastos no cartão de crédito em função do gênero

***ANCOVA – salários em função de faixa etária, gênero, anos de empresa



EXEMPLO

Id.	Nome	Idade	Sexo	Peso	Manchas	Temp.	# Int.	Est.	Diagnóstico
4201	João	28	M	79	Concentradas	38,0	2	SP	Doente
3217	Maria	18	F	67	Inexistentes	39,5	4	MG	Doente
4039	Luiz	49	M	92	Espalhadas	38,0	2	RS	Saudável
1920	José	18	M	43	Inexistentes	38,5	8	MG	Doente
4340	Cláudia	21	F	52	Uniformes	37,6	1	PE	Saudável
2301	Ana	22	F	72	Inexistentes	38,0	3	RJ	Doente
1322	Marta	19	F	87	Espalhadas	39,0	6	AM	Doente
3027	Paulo	34	M	67	Uniformes	38,4	2	GO	Saudável

AS VÁRIAS POSSIBILIDADES E TÉCNICAS

Resumo: exemplos de algoritmos

Gerais: Árvore de decisão (decision tree), floresta aleatória (Random forest), redes neurais, gradient boosting

Classificadores OU Preditores (regressores)

Específicos:

Regressão linear (numérico, numérico – contínuos)

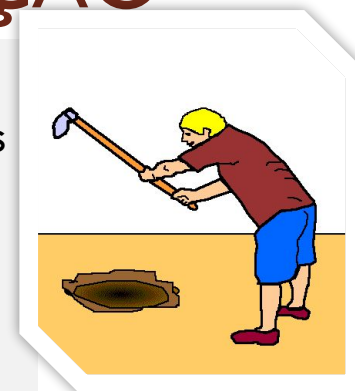
Regressão logística (qualquer X, mas Y categórico/qualitativo) – muito usado em diagnóstico

Indução de regras (rule induction) – independente do dado de entrada, extrair regras if-then-else
Algoritmos como lem1, lem2, aq etc.

ANOVA e ANCOVA são análises de variância, um tipo de teste de hipóteses. Ambos tem variável target numérica, mas ANOVA tem X apenas categórico, e ANCOVA X pode ser misto

RETORNANDO AO CICLO DO DADO: **PRODUÇÃO**

- 1) Dados podem ser coletados, **produzidos** ou simplesmente comprados (*data brokers*)
 - 1) PRODUZIDOS como resultado de processamento (Ex. folha de pagamento) em sistemas transacionais,
 - 2) podem ser resultado de TRANSFORMAÇÃO de dados (para análise, armazenamento)
 - 3) podem ser produzidos por modelagem estatística ou aprendizagem de máquina
 - 3) podem ser COLETADOS de outros sistemas, pesquisas, dados históricos, arquivos, de um Data Warehouse
 - 4) Das chaves mecânicas, passando pelos cartões perfurados, às telas touch-screen e sensores de movimento
 - 5) teclados, mouses, leitores (barras, QR), RFID, mesas digitalizadoras, câmeras
 - 6) Projetos de computação distribuída, com base em doação de tempo de CPU ([SETI](#), [CLIMATE PREDICTION](#), [ROSETTA@](#))



Acelerômetros – aceleração do objeto em 3 eixos
GPS

Giroscópio – orientação do celular em 3 eixos

Magnetômetro – mede o campo magnético da Terra

Podem se utilizar das interfaces de comunicação:

Wifi, bluetooth, nfc etc.

WEARABLES

Em Desenvolvimento APIs permitem manipulação

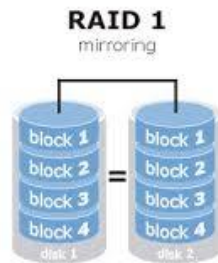


CC BY-SA-NC



RETORNANDO AO CICLO DO DADO: **ARMAZENAMENTO**

- 1) Uma vez armazenado, pode ser recuperado para replicar processos ou produzir informação ou conhecimento
- 2) Contempla as premissas de segurança da informação, integridade, minimização da redundância, concorrência, otimização de espaço etc.
- 3) Podem ser simplesmente REPLICADOS em sua forma original (backup, redundância (RAID 1, clusterização)
- 4) Já num data warehouse, é armazenado com sua estrutura modificada



RETORNANDO AO CICLO DO DADO: **ARMAZENAMENTO**

RAID é um conceito de armazenamento em nível de bloco/hardware, enquanto o MapReduce é um paradigma de processamento de software.

No entanto, o conceito de "**armazenamento distribuído com paridade e tolerância a falhas**" (que é a ideia central do RAID) foi adaptado para a era do Big Data e do MapReduce na forma de **Erasure Coding em Sistemas de Arquivos Distribuídos**.

As tecnologias que unem a filosofia de tolerância a falhas do RAID com ecossistemas baseados em MapReduce são:

RETORNANDO AO CICLO DO DADO: **ARMAZENAMENTO**

HDFS Erasure Coding (EC) — O "RAID do Hadoop"

O **HDFS (Hadoop Distributed File System)** é o sistema de arquivos que alimenta o MapReduce original.

- **Antes:** Para não perder dados se uma máquina queimasse, o HDFS simplesmente triplicava tudo (fator de replicação 3x). Isso gastava 200% a mais de espaço em disco.
- **Com Erasure Coding:** A partir do [Hadoop 3.0](#), o HDFS passou a implementar algoritmos como **Reed-Solomon** (o mesmo tipo de matemática usada em variações avançadas de RAID). O HDFS fatia os blocos de dados entre vários nós do cluster e gera blocos de paridade distribuídos.
- **Resultado:** Ele funciona exatamente como um **RAID 5 ou RAID 6 em escala de cluster**, economizando até 50% de espaço em disco enquanto mantém o suporte nativo ao MapReduce.

MODELOS DE ARMAZENAMENTO

A Medida que o volume e complexidade aumentaram, a partir de 1950 surge necessidade de gerenciadores capazes de incluir, alterar e excluir, mas também de manter integridade, segurança e de prover indexação.

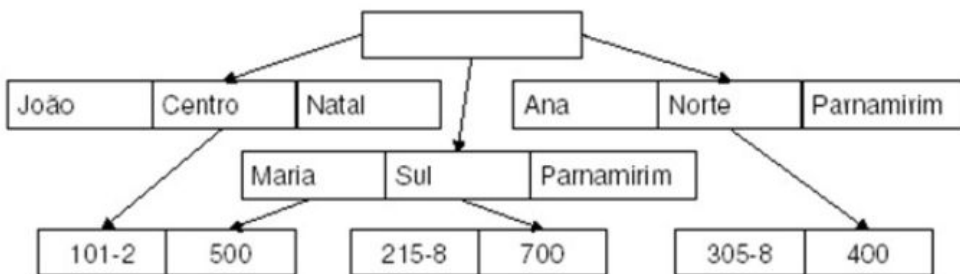
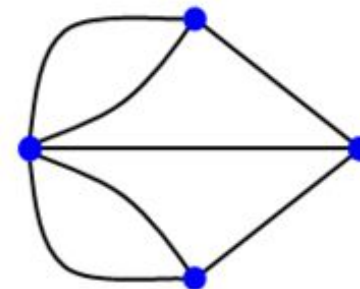
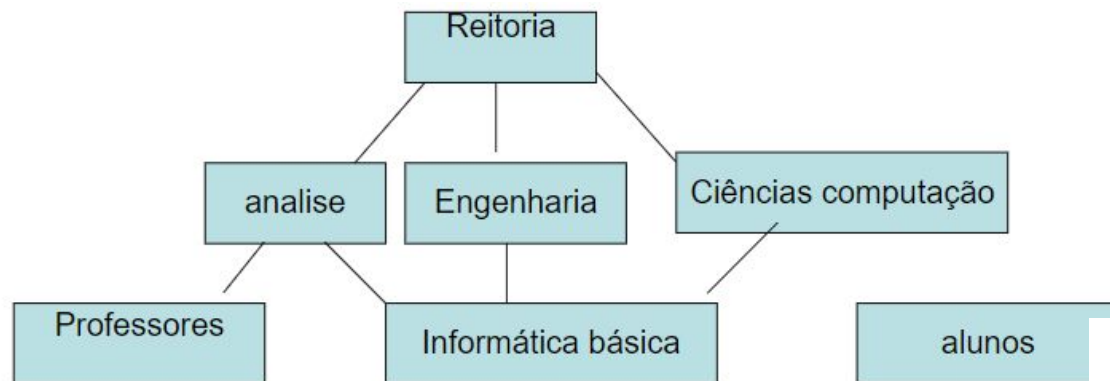
Modelo	Década
Pré-relacionais (hierárquico, rede (codasyl))	60
Relacional	70
<i>Orientado a objetos (caché, Db4o, Versant...)</i>	80
NoSQL (kvs, colunas ordenadas, documentos, grafos)	2000
NoSQL vetoriais (embeddings) (Pinecone , Milvus ou Qdrant , pgvector, etc.)	2011 (força 2022 em diante)

MODELOS DE ARMAZENAMENTO

Modelo de rede baseado em navegação – registros vinculados precisavam ser percorridos numa árvore, Unidirecional, do pai para o filho, não havendo controle efetivo de integridade e redundância

O consórcio CODASYL é conhecido por ter criado a linguagem COBOL relacionamentos: LINKS

OBS: *ideia de Grafo e sua teoria remonta a Leonhard Euler (1736) e as pontes de Königsberg*



MODELOS DE ARMAZENAMENTO

RELACIONAL: Criado por Edgar Codd em 1970 (*Relational Model of Data for Large Shared Data Banks*)

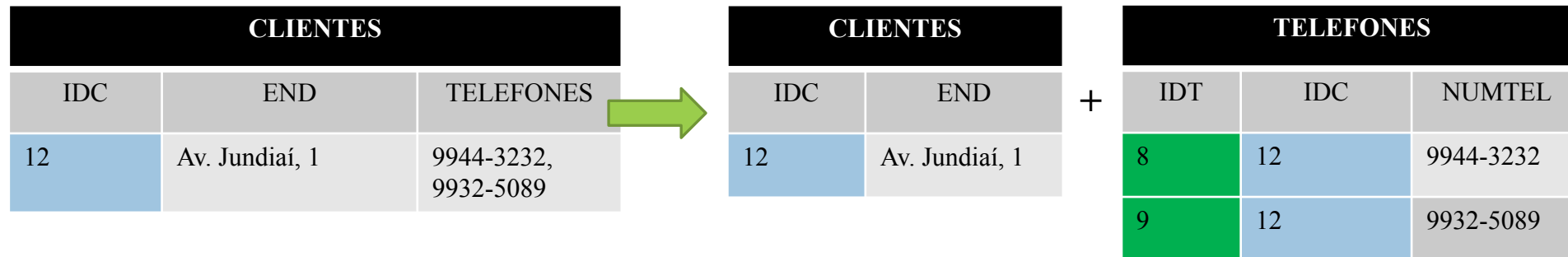
Foco na manutenção da integridade das transações (*inclui, altera, exclui*) e redução da redundância

- 1) Baseado em álgebra relacional (junções), abstrai implementação física do BD, permitindo realizar consultas através da Linguagem Estruturada de Consulta (SQL) – recuperar dados distribuídos em várias TABELAS
- 2) Estrutura básica (rígida e fixa): TABELAS, com instâncias (registros) sendo as linhas (row) e os atributos as colunas (column), (schema) 
- 3) Bom para armazenamento, não necessariamente para ANÁLISE (podem haver consultas complexas)
- 4) Relacionamentos são mantidos por identificadores únicos denominados CHAVES: primária (PK) e estrangeira (FK), possuindo o conceito de cardinalidade: um-para-um, um-para-muitos, muitos-para-muitos,...
- 5) Implementações confiáveis, estáveis, eficientes, permitindo backups incrementais, replicação, clusterização, tolerância às falhas, distribuição de carga entre outros: DB2, Oracle, SQL Server, PostgreSQL, MySQL etc. SQLite é exceção, embora relacional, não exige tipagem no schema e, mesmo que haja, aceita tipo diferente, pois é baseado no tipo do valor e não da coluna

MODELOS DE ARMAZENAMENTO

RELACIONAL: Criado por Edgar Codd em 1970 (*Relational Model of Data for Large Shared Data Banks*)

Foco na manutenção da integridade das transações (*inclui, altera, exclui*) e redução da redundância



6) CODD definiu a NORMALIZAÇÃO e suas FORMAS NORMAIS (FN)

6) 1FN: atributos atômicos, sem valores repetidos ou possuindo mais de um valor. Por exemplo, CLIENTE = {IDC + END + TELEFONES}, como TELEFONES é multivalorado, ou remove ou cria uma outra TABELA com IDC como chave ESTRANGEIRA

MODELOS DE ARMAZENAMENTO

6.2) 2FN: os atributos não chave devem depender unicamente da PK da tabela em questão. As colunas que não são dependentes apenas da PK são removidas e colocadas em outra tabela

professorCurso = {idProf, idCurso, salario, descricaoCurso}: descricaoCurso não depende de idProf, mas de idCurso

professorCurso				→	professorCurso			+	cursos	
idProf	idCurso	salario	descricaoCurso		idProf	idCurso	salario		idCurso	descricaoCurso
12	10	1.500,00	computação		12	10	1.500,00		10	computação

6.3) 3FN: projetada para melhorar desempenho do BD e minimizar custos de armazenamento. Atributos devem ser independentes funcionalmente uns dos outros.

funcionarios = {idFunc, nome, salario, fgts}: fgts (não chave) depende de salario (não chave), logo pode ser retirado e deixar o cálculo para a camada de negócio ou, se necessário, *ir para outra tabela referenciando funcionários*.

MODELOS DE ARMAZENAMENTO

6.4) 4FN: remover multiplicidade de campos multivalorados

Solicitação de Exame

Paciente	Plano de Saúde	Exame
João	Amil	Teor alcoólico
João	Blue-Life	Teor alcoólico
João	Amil	Sangue
João	Blue-Life	Sangue



Solicitação de Exame

<u>Paciente</u>	<u>Exame</u>
João	Teor alcoólico
João	Sangue

Suporte de Seguridade

<u>Paciente</u>	<u>Plano de Saúde</u>
João	Amil
João	Blue-Life

MODELOS DE ARMAZENAMENTO

6.4) 4FN: remover multiplicidade de campos multivalorados

ID_CIDADE (PK)	CIDADE
1	Brasília
2	Rio de Janeiro
3	Campinas

Tabela 8 – Exemplo de 4a. forma normal
(relação de cidades).

ID_ANO (PK)	ANO_INGRESSO
1	2014
2	2017
3	2018

Tabela 9 – Exemplo de 4a. forma normal
(relação de anos).

ID_PROFESSOR (PK)	NOME	ID_CIDADE (FK)	ID_ANO (FK)
1	Thiago Cavalcanti	1	1
2	Renato da Costa	2	3
3	Felipe Luccas	3	2
4	Adriana Figueiredo	3	2
5	André Castro	1	1

Tabela 10 – Exemplo de 4a. forma normal (relação de professores).

<https://www.estrategiaconcursos.com.br/blog/banco-dados-forma-normal/>

MODELOS DE ARMAZENAMENTO

7) A PK pode ter valor semântico ou incremental e também pode ser simples ou composta

DESNORMALIZADO

Cliente	Cliente ID	Transação		
João	1	Tr. ID	Data	Valor
		12890	14/out/2003	-87
		12904	15/out/2003	-50
Wilson	2	Tr. ID	Data	Valor
		12898	14/out/2003	-21
Márcio	3	Tr. ID	Data	Valor
		12907	15/out/2003	-18
		14920	20/nov/2003	-70
		15003	27/nov/2003	-60



Tabela dos clientes

Cliente	Cliente ID
João	1
Wilson	2
Márcio	3

Tabela das transações

Cliente ID	Tr. ID	Data	Valor
1	12890	14/out/2003	-87
1	12904	15/out/2003	-50
2	12898	14/out/2003	-21
3	12907	15/out/2003	-18
3	14920	20/nov/2003	-70
3	15003	27/nov/2003	-60

```
SELECT Transacoes.TrID, Clientes.Cliente
FROM Transacoes
INNER JOIN Clientes ON Transacoes.ClienteID = Clientes.ClienteID;
```

MODELOS DE ARMAZENAMENTO

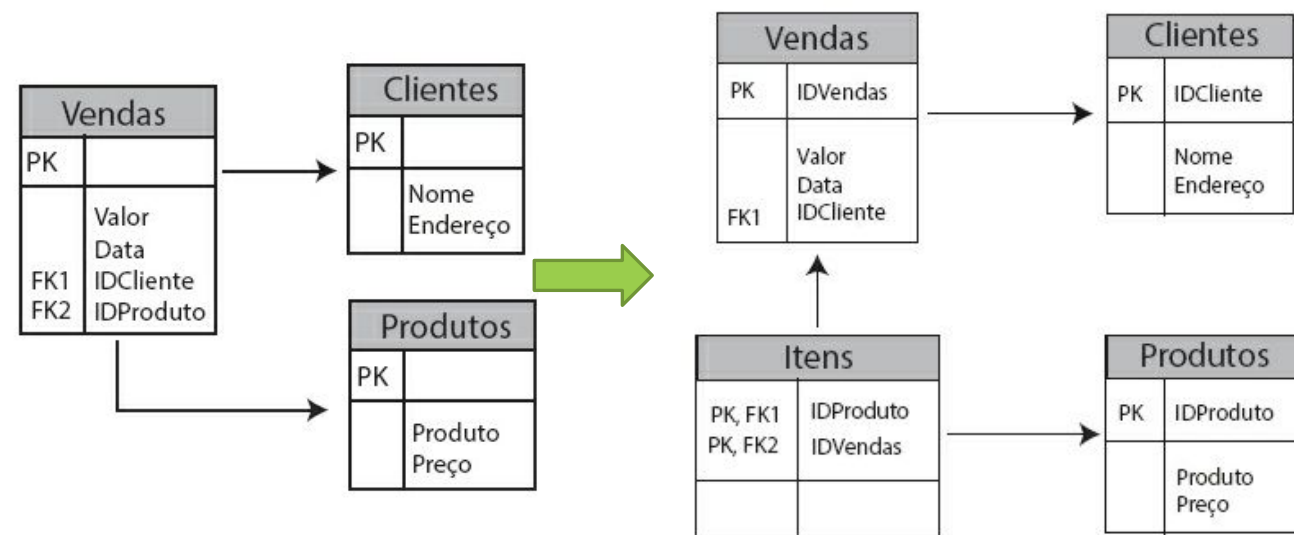
7) A PK pode ter valor semântico ou incremental e também pode ser simples ou composta

Produto e cliente repetidos, não se sabe ao certo se é o mesmo cliente
Como o valor não tem referência única, qual linha tem o valor atual?
Já imaginou a cada compra cadastrar os dados do cliente?

DESNORMALIZADO

Vendas	
Cliente	
Endereço	
Produto	
Valor	
Data	

Vendas				
Cliente	Endereço	Produto	Valor	Data
José da Silva	Bloco A, Casa 3	Chupeta	5,34	12/09/2014
Maria Cardoso	Rua Brasil, 44	Mamadeira	12,30	13/09/2014
Pedro Henrique	Bloco C, casa 24	Colchão	348,00	12/09/2014
José da Silva	Bloco A, casa 3	Chupeta	6,00	13/09/2014
Pedro H.	Bloco C, casa 24	Mamadeira	12,30	13/09/2014



MODELOS DE ARMAZENAMENTO



OBSERVAÇÕES AO MODELO RELACIONAL

- De fato o modelo apresenta boa integridade, reduz redundância, mas possui muitas TABELAS
- Torna assim mais complexa a álgebra das consultas para recuperar dados (para analisar)
- Com o aumento do volume de dados, operações demandam custo computacional elevado
- O desenvolvimento da web passa a demandar armazenamento de dados não estruturados, com escalabilidade e redundância – a partir dos anos 90, o volume aumenta e a dificuldade em analisar conseqüentemente
- Organizações tinham vários bancos de dados separados e necessitavam agregar estes dados e transformá-los para as operações de análise – surgem os DATA WAREHOUSES
- Os modelos NewSQL ou NoSQL surgem para atender a tais situações, não substituindo o relacional, mas sendo uma alternativa a depender do problema. Para aplicações com demandas de integridade, informações estruturadas e normalizadas, o relacional ainda tem seu espaço
- O NoSQL usa um modelo mais simples, de objetos com atributos, os quais trazem flexibilidade na definição e cada objeto da coleção pode trazer apenas um subconjunto de atributos
- Adequado para situações onde não é fácil ou possível decompor em conjunto de atributos, como por exemplo texto livre (tweets etc.) ou imagens – contudo necessitam serem colocados em algo estruturado para análise

NOSQL (NOT ONLY SQL)

- Aplicações para agregar grandes volumes de dados (gestão de documentos, séries temporais, feed de notícias, dados em painéis em tempo real)
- Problema não só de R/W, mas o simples armazenamento em si
- Como armazena-se +1bi de usuários do Facebook e o conteúdo que gerem diariamente?
- Produtos baseados em não normalização, poucas restrições de integridade e controle mínimo de transações
- Contudo também podem processar dados altamente estruturados

NoSQL	Relacional
Esquema	
Não há necessidade de um esquema fixo, o que dá uma maior liberdade de armazenamento.	Tem que ser definido antes de qualquer operação, limitando o armazenamento.
Relação	
Sem relações, a informação é armazenada como um agregado, onde um único arquivo possui tudo sobre a transação.	As relações são estabelecidas por conexões entre tabelas.
Distribuição	
Múltiplos computadores podem armazenar dados de uma mesma base de dados.	Múltiplos computadores podem armazenar e processar dados, porém se especificados.

Ver como fazer relacionamentos no mongodb!



mySQL

```
SELECT
  Dim1, Dim2,
  SUM(Measure1) AS MSum,
  COUNT(*) AS RecordCount,
  AVG(Measure2) AS MAvg,
  MIN(Measure1) AS MMin
  MAX(CASE
    WHEN Measure2 < 100
    THEN Measure2
  END) AS MMax
FROM DenormAggTable
WHERE (Filter1 IN ('A','B'))
  AND (Filter2 = 'C')
  AND (Filter3 > 123)
GROUP BY Dim1, Dim2
HAVING (MMin > 0)
ORDER BY RecordCount DESC
LIMIT 4, 8
```

- ① Grouped dimension columns are pulled out as keys in the map function, reducing the size of the working set.
- ② Measures must be manually aggregated.
- ③ Aggregates depending on record counts must wait until finalization.
- ④ Measures can use procedural logic.
- ⑤ Filters have an ORM/ActiveRecord-looking style.
- ⑥ Aggregate filtering must be applied to the result set, not in the map/reduce.
- ⑦ Ascending: 1; Descending: -1

MongoDB

```
db.runCommand({
  mapreduce: "DenormAggCollection",
  query: {
    filter1: { '$in': [ 'A', 'B' ] },
    filter2: 'C',
    filter3: { '$gt': 123 }
  },
  map: function() { emit(
    { d1: this.Dim1, d2: this.Dim2 },
    { msum: this.measure1, recs: 1, mmin: this.measure1,
      mmax: this.measure2 < 100 ? this.measure2 : 0 }
  );},
  reduce: function(key, vals) {
    var ret = { msum: 0, recs: 0, mmin: 0, mmax: 0 };
    for(var i = 0; i < vals.length; i++) {
      ret.msum += vals[i].msum;
      ret.recs += vals[i].recs;
      if(vals[i].mmin < ret.mmin) ret.mmin = vals[i].mmin;
      if((vals[i].mmax < 100) && (vals[i].mmax > ret.mmax))
        ret.mmax = vals[i].mmax;
    }
    return ret;
  },
  finalize: function(key, val) {
    val.mavg = val.msum / val.recs;
    return val;
  },
  out: 'result1',
  verbose: true
});
db.result1.
  find({ mmin: { '$gt': 0 } }).
  sort({ recs: -1 }).
  skip(4).
  limit(8);
```

NOSQL - KVS

- TIPOS:
- **Key-Value Store** (KVS) – (chave-valor)
todos os registros fazem parte da mesma coleção de elementos, e a única coisa que todos eles tem em comum é a chave única; invés de incluir um conjunto de atributos, a operação insere apenas uma chave e um valor (Couchbase, Kyoto Cabinet, **Redis** (open source), **DynamoDB** - Amazon) – **BD as a service**
- **Estrutura mais SIMPLES**
- **Grande tabela hash (pareamento)**
- Base do tipo DOCUMENTOS – grupo de coleções

Chaves	Outros atributos		
a	colA:value1	colFoo:a value	fram:zilk
b	colA:value1	colB:a value	♟: chesspiece
bb	colA:value1	colB: colFoo:a value	🎵: 🎵
c	colA:☺	colBaz:anything	colFoo:a value

Chave	Valor
16	nome = NoSQL Essencial, ano - 2014
Connor	idade = 22, interesse = programação
2	nome = True Blood, gênero = fantasia, classificação = 16 anos
Laís	ocupação = estudante

new item

new item

New item



```
import redis
r = redis.Redis(
    host= 'redis.us-east-1-1.ec2.cloud.redislabs.com',
    port= '12345',
    Password = 'thisismypassword')
r.set('foo','bar')
r.get('foo')
```

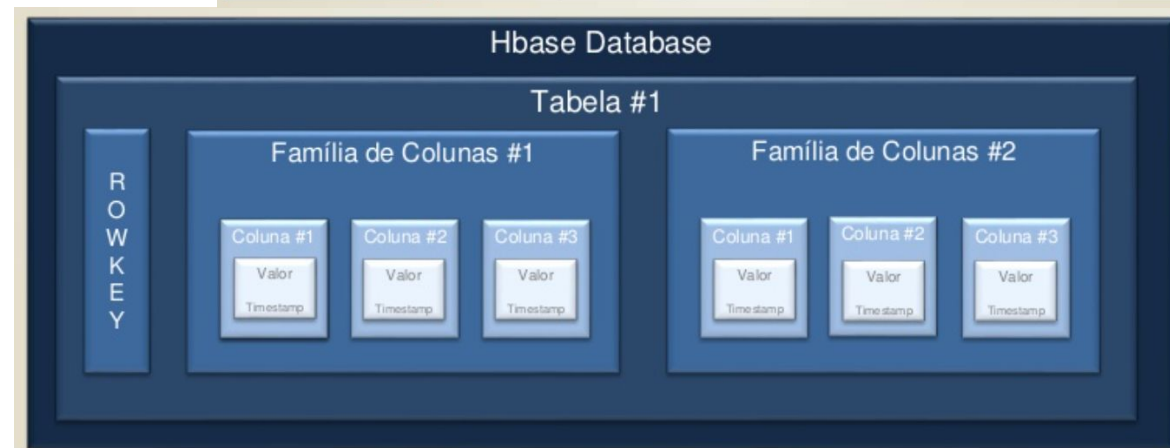
NOSQL - COLUNAR



- TIPOS:
- **Colunas Ordenadas (Colunar)**
- Baseado no modelo BigTable do Google, onde o dado é **orientado por coluna** ao invés de linha. São implementações deste modelo o Hbase (Apache), o HyperTable da Clouddata
- Bancos de dados colunares, de coluna larga ou de famílias de colunas armazenam dados de modo eficiente, consultam linhas de dados esparsos são vantajosos ao consultar em colunas específicas no banco de dados.
- Projeto baseado no que se quer consultar, quais dados são obtidos juntos?
- Desnormalização: evitar JOINS – todos os dados da app numa linha ou conjunto de linhas (regiões – faixas contínuas de rowkeys)

RowKey	FC1			FC2		
	col1	col2	col3	col1	col2	col3
row1	Valor		Valor		Valor	Valor
row2		Valor		Valor		
row3	Valor	Valor		Valor		Valor
row4			Valor		Valor	
row5	Valor					Valor
row6	Valor	Valor	Valor	Valor	Valor	Valor

Roda sobre o HDFS, integrado ao Hadoop e escala linearmente para lidar com grandes conjuntos de dados com bilhões de linhas e milhões de colunas e combina facilmente fontes de dados que utilizam uma grande variedade de estruturas e esquemas diferentes.



NOSQL - COLUNAR

RowKey	Blog:t1	Blog:t2		Blog:t3
userid1	A Amazônia está	O cenário econômico		O Campeonato carioca
userid2	Receita de bolos ..	Sobremesas Diet		Pratos leves ...
userid3	Projetos de Arduino	Conhecendo IOT		
userid4	Esportes radicais ...	Triathlon		Stand up Paddle

Estas chaves normalmente estão distribuídas no nós da rede
Desnormalizar neste caso acelera leitura mas é custoso para UPDATE

- CRUD, com operações de CREATE TABLE, PUT, GET, SCAN
- Formas de acesso:
 - Hbase shell – (em JRuby)
 - API Java, REST, integração com Spark, Hive, Pig

```
hbase(main):002:0> create 'tab1', 'familia1', 'familia2'
0 row(s) in 1.3390 seconds
```

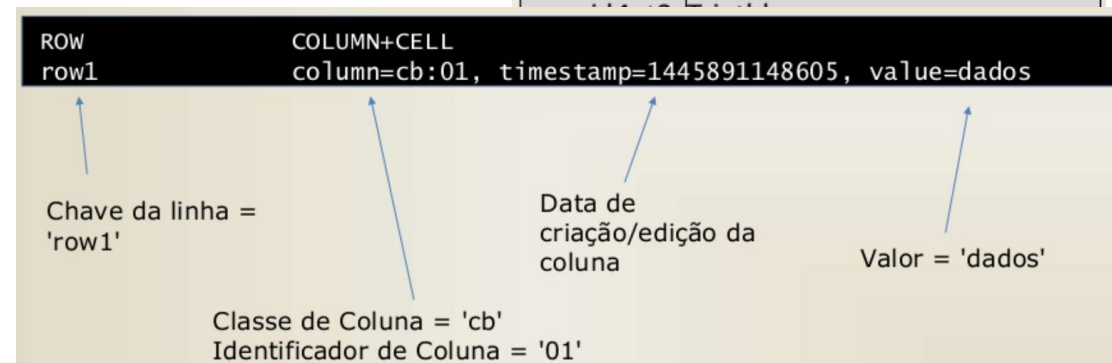
```
=> HBase::Table - tab1
```

```
hbase(main):010:0> put 'tab1', 'linha1', 'familia1:colunaA', 'aaaaa'
0 row(s) in 0.0750 seconds
```

```
hbase(main):002:0> get 'test', 'row1'
COLUMN                                CELL
cb:01                                timestamp=1445891148605,
value=dados
1 row(s) in 0.0070 seconds
```

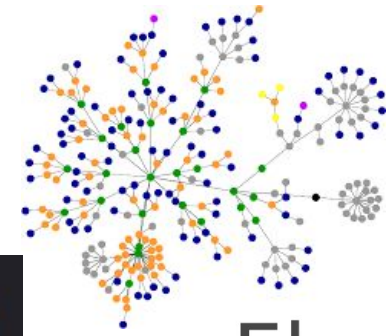
- 1) Todos os blogs armazenados numa única RowKey (modelo Flat-Wide, poucas linhas muitas colunas); Aqui cada blog tem em seu valor o timestamp.
- 2) oposto ao Tall-narrow, neste caso, com um blog por linha, e o ROWKEY é o userID+timestamp (+detalhe)

RowKey	Blog
userid1_t1	A Amazônia está
userid1_t2	O cenário econômico
userid1_t3	O Campeonato carioca
userid2_t1	Receita de bolos ..
userid2_t2	Sobremesas Diet
userid2_t3	Pratos leves ...
userid3_t1	Projetos de Arduino
userid3_t2	Conhecendo IOT
userid3_t3	Raspberry PI 3
userid4_t1	Esportes radicais ...

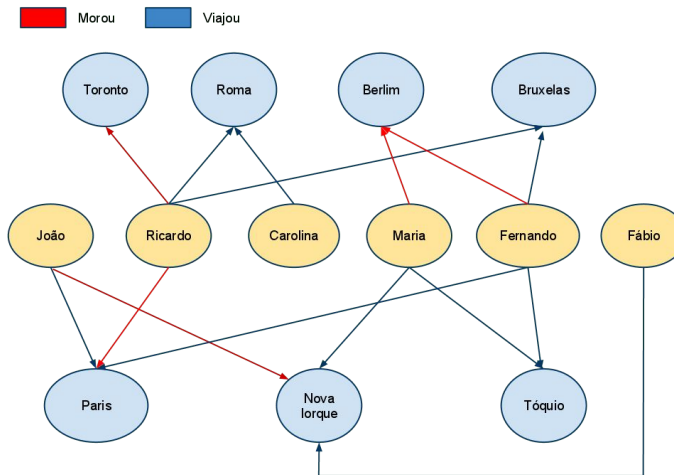
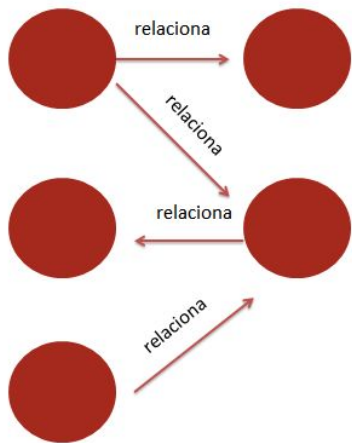


NOSQL - GRAFOS

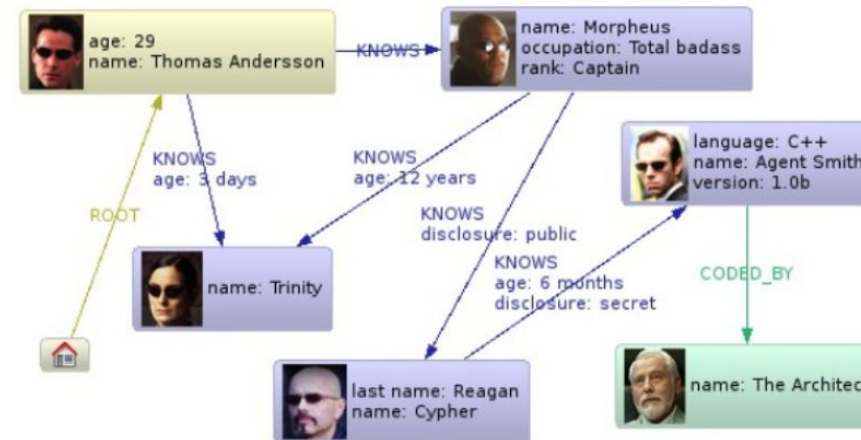
- A grande diferença para o modelo clássico está na representação explícita de relacionamentos entre os **dados**, através de um modelo com vértices, chamados de **nós**, e arcos ou arestas, chamados de **relações**
- Para representar relacionamentos complexos, o grafo é eficaz, em parte pela eficiência de seus algoritmos de BUSCA
- Suas inúmeras aplicações na medicina, computação, genética, economia, matemática abstraem os algoritmos e implementações



FlockDB

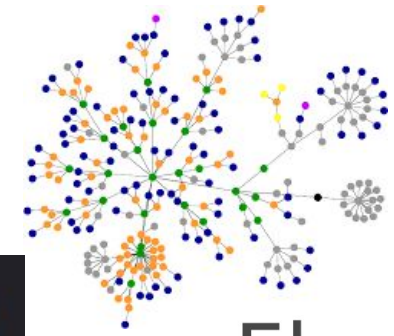


- 1) Nós e arestas possuem ATRIBUTOS
- 2) Relacionamentos possuem DIREÇÃO
- 3) São particularmente interessantes em semântica WEB



NOSQL - GRAFOS

- Itens de um BD de Grafos
 - Vértice (Nó) – conjunto de propriedades chave:valor que representam uma entidade. Exemplo, um usuário do Twitter
nome: 'Joaquim Miguel'
arroba: 'Trinity'
 - Arestas – são os relacionamentos. Ligam os vértices por meio de ligações semânticas. Exemplo baseado no Twitter: nó 'Priscila' segue o nó 'Lucas' desde 2012 – o nome do relacionamento é 'segue', que possui um sentido: 'Priscila'-'>'Lucas' e dados: data de início do relacionamento
 - Usualmente acesso por API REST; no caso do neo4j, linguagem **Cypher**



FlockDB



AllegroGraph, ArangoDb, Bitsy, DEX, InfiniteGraph, InfoGrid, Oracle Spatial, HyperGraphDB, Titan...

Comparativo postgresql/node4js

[DB-Engines Ranking - popularity ranking of database management systems \(db-engines.com\)](http://db-engines.com)

NOSQL - GRAFOS

Tabela 1. Comparação sintática entre SQL e Cypher.

Cláusula	SQL	Cypher
Seleção	<code>SELECT * FROM usuario;</code>	<code>MATCH (u:usuario) RETURN u;</code>
Filtragem	<code>SELECT * FROM usuario WHERE username LIKE 'alvstricklan8196';</code>	<code>MATCH (u:usuario) WHERE u.username = 'alvstricklan8196' RETURN u;</code>
Junção	<code>SELECT * FROM usuario u JOIN post p ON u.username = p.usuario;</code>	<code>MATCH c=(u:usuario)-[:PUBLICA]->(p:post) RETURN c;</code>
Agrupamento	<code>SELECT u.username, count(*) FROM usuario u JOIN post p ON u.username = p.usuario GROUP BY u.username;</code>	<code>MATCH (u:usuario)-[:PUBLICA]->(p:post) RETURN u.username, count(u);</code>
Ordenação	<code>SELECT * FROM usuario u ORDER BY u.nome DESC;</code>	<code>MATCH (u:usuario) RETURN u ORDER BY u.nome DESC;</code>

```
start programmer=(3)
match (programmer)-[:PAIRED]->(pair)
where pair.age > 30
return pair order by pair.age skip 5 limit 10
```

Cypher

CONSOLIDAÇÃO

- Construir tabela comparativa com bancos de dados relacionais, chave valor, de documentos, grafos, vetoriais, colunares, destacando:
 - 3 exemplos comerciais de cada (do mais usado para o menos usado)
 - sugestão de aplicações, contextos e problemas interessantes de usar
 - como se faz busca: sql ou outra linguagem